

BAB IV

PENGENALAN MERN STACK

4.1 Tujuan Pembelajaran

1. Memahami konsep arsitektur aplikasi web full-stack.
2. Menjelaskan peran masing-masing teknologi dalam MERN Stack.
3. Memahami fungsi:
 - Node.js sebagai runtime server-side
 - Express.js sebagai backend framework
 - React sebagai frontend library
 - MySQL sebagai sistem manajemen basis data
4. Membuat REST API sederhana menggunakan Express.
5. Menghubungkan backend dengan database MySQL.
6. Menghubungkan frontend React dengan backend melalui HTTP request.
7. Memahami alur data client-server

4.2 Dasar Teori

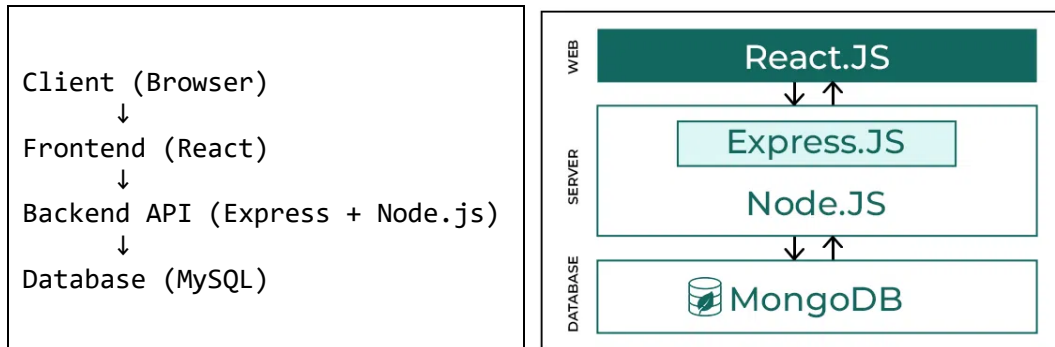
4.2.1. Konsep Full-Stack Development (MERN STACK)

MERN Stack adalah salah satu kombinasi teknologi (*tech stack*) paling populer di dunia pengembangan web untuk membangun aplikasi *Fullstack* yang modern, dinamis, dan skalabel. Istilah MERN merupakan akronim dari empat teknologi utama berbasis JavaScript yang digunakan di dalamnya, yaitu **M**ongodb, **E**xpress, **R**ead dan **N**ode. Namun dalam media pembelajaran kali ini penggunaan *MongoDB* akan digantikan dengan *MySQL* sebagai media penyimpanan data.

Full-stack development adalah proses pengembangan aplikasi yang mencakup:

- ✓ Frontend → Antarmuka pengguna
- ✓ Backend → Logika dan pengolahan data
- ✓ Database → Penyimpanan data

Arsitektur umum:



Mengapa MERN Stack Sangat Populer?

- Satu Bahasa (JavaScript Everywhere): Pengembang tidak perlu belajar banyak bahasa. Cukup kuasai JavaScript, bisa menjadi Fullstack Developer.
- Kecepatan Pengembangan: Banyaknya library pendukung membuat proses pembuatan aplikasi menjadi sangat cepat.

- Performa Tinggi: Aplikasi MERN dikenal sangat responsif (seperti aplikasi mobile tapi di web).

4.2.2. MySQL

MySQL adalah sistem manajemen basis data relasional (RDBMS).

Karakteristik:

- Menggunakan SQL
- Tabel, relasi, *primary key*, *foreign key*
- Cocok untuk sistem terstruktur

Query:

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100)
);
```

4.2.3. Express JS

Express.js adalah framework minimalis untuk Node.js yang mempermudah pembuatan server dan API.

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("Server berjalan");
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Fungsi utama:

- Routing
- Middleware
- REST API
- Handling request & response

4.2.4. React JS

React adalah library JavaScript untuk membangun antarmuka pengguna berbasis komponen.

Karakteristik:

- Component-based
- Virtual DOM
- Reusable
- Reactive state

Contoh komponen sederhana:

```
import React from 'react'

function Wellcome() {
```

```

return (
  <div>
    <h1>Hello world!!</h1>
  </div>
)
}

export default Wellcome;

```

React berfungsi sebagai:

- Frontend UI
- Pengelola state
- Pengirim request ke backend (axios)

4.2.5. Node JS

Node.js adalah runtime JavaScript yang memungkinkan JavaScript dijalankan di sisi server.

Fungsi:

- Menjalankan server
- Mengelola request & response
- Menghubungkan ke database
- Mengelola API

Keunggulan:

- Non-blocking (asynchronous)
- Cepat dan ringan
- Satu bahasa untuk frontend & backend

4.2.6. REST API dan HTTP Method

REST API adalah antarmuka komunikasi antara frontend dan backend.

Method	Fungsi
GET	Mengambil data
POST	Menambahkan data
PUT	Mengupdate data
DELETE	Menghapus data

Contoh endpoint:

GET	/users
POST	/users
PUT	/users/:id
DELETE	/users/:id

4.3 Latihan Pembelajaran

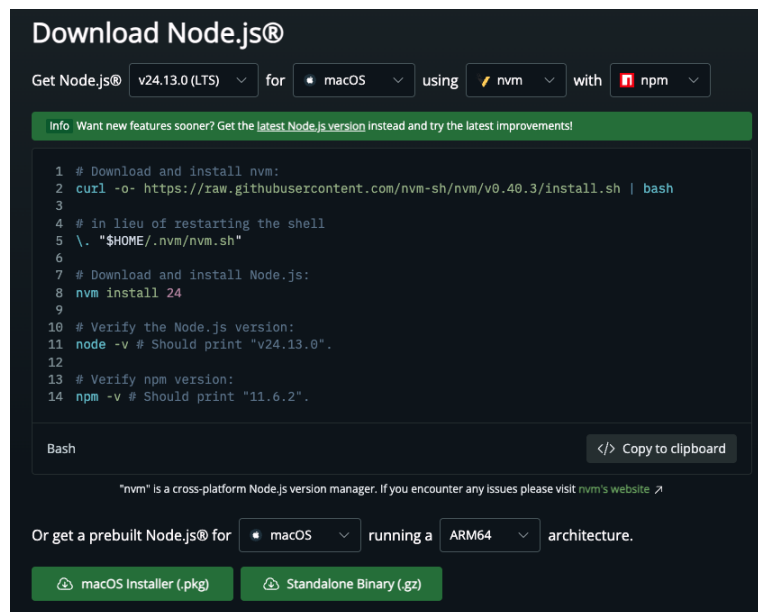
Pada latihan pembelajaran BAB ini, akan memulai proses pembangunan aplikasi web berbasis *Full-Stack JavaScript* menggunakan pendekatan MERN Stack (*MySQL, Express, React, Node*).

Latihan ini dirancang secara bertahap, dimulai dari tahap instalasi perangkat lunak yang dibutuhkan, konfigurasi lingkungan pengembangan (*development environment*), hingga implementasi aplikasi sederhana yang terintegrasi antara *frontend*, *backend*, dan database.

- Node.js digunakan untuk menjalankan server.
- Express.js digunakan untuk membangun REST API.
- MySQL digunakan untuk menyimpan dan mengelola data.
- React digunakan untuk membangun antarmuka pengguna (UI).

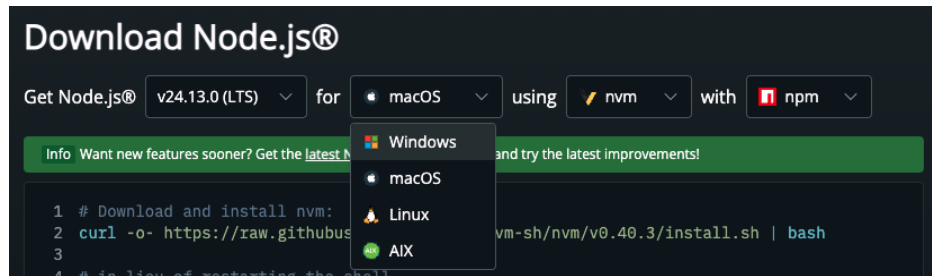
Langkah-1: Instalasi dan Persiapan Lingkungan

1. Buka halaman resmi Node.js (LTS version).

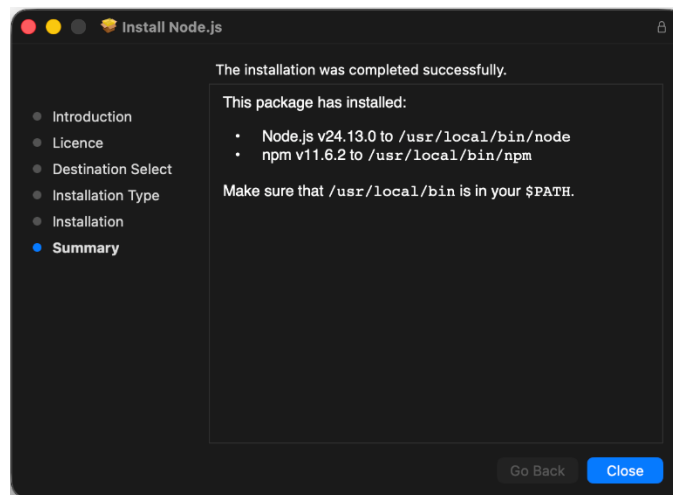


(<https://nodejs.org/en/download>)

2. Download dan install sesuai sistem operasi.



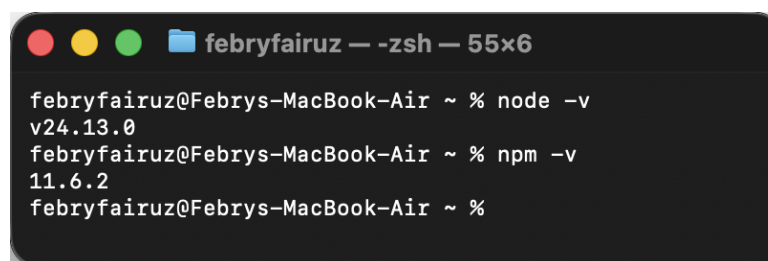
Setelah mendownload sesuai dengan Sistem Operasi maka silakan install package tersebut.



Gambar Instalasi Package Node JS

3. Setelah selesai, buka Terminal / Command Prompt:

Buka terminal atau command prompt pada komputer dan masukan syntax dibawah ini: `node -v` dan `npm -v` secara terpisah untuk mengetahui bahwa telah berhasil dan menggunakan versi yang sesuai.

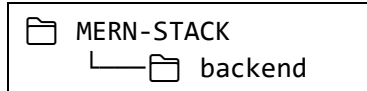


Note:

Updated version of node @ Feb 2026

Langkah-2: Membangun Backend Server

1. Buatlah folder bernama **MERN-STACK** lalu didalamnya buatlah folder **'backend'** seperti sturktur dibawah ini:



2. Buka terminal dan masukan syntax ini secara bergantian pada directory backend:

```
npm init -y  
npm install express mysql2 cors
```

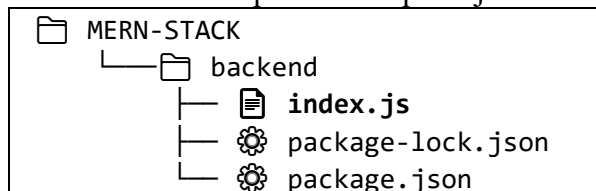
Syntax pertama akan membuat package.json sebagai bagian konfigurasi server node bernama backend:

```
D:\25-26 Genap\Pem Web\wokspaces\MERN-STACK\backend>npm init -y  
Wrote to D:\25-26 Genap\Pem Web\wokspaces\MERN-STACK\backend\package.json:  
  
{  
  "name": "backend",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC"  
}
```

Sedangkan syntax yang kedua akan menginstall tiga buah library package yaitu express mysql2 dan cors:

```
D:\25-26 Genap\Pem Web\wokspaces\MERN-STACK\backend>npm install express mysql2 cors  
  
added 76 packages, and audited 77 packages in 5s  
  
25 packages are looking for funding  
  run `npm fund` for details  
  
found 0 vulnerabilities  
  
D:\25-26 Genap\Pem Web\wokspaces\MERN-STACK\backend>
```

3. Buatlah file index.js untuk membuat script server express js



File index.js

```
const express = require("express");  
const app = express();
```

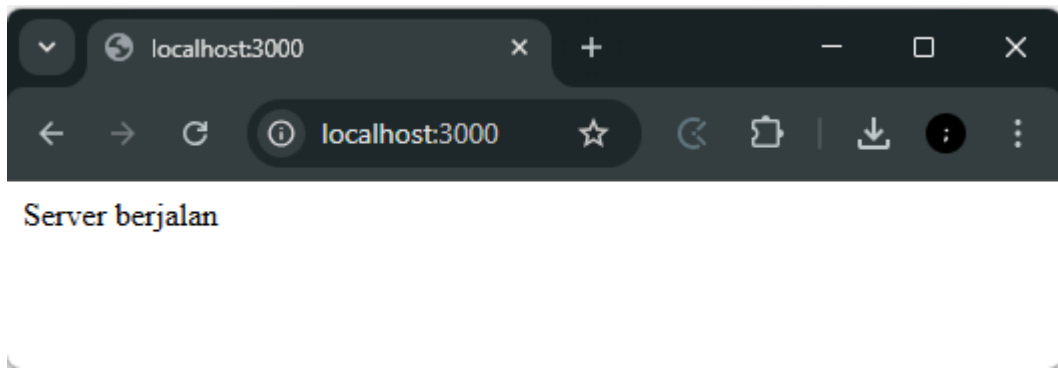
```
app.get("/", (req, res) => {  
  res.send("Server berjalan");  
});  
  
app.listen(3000, () => {  
  console.log("Server running on port 3000");  
});
```

Cara menjalankannya buka terminal masuk kedalam directory path backend dan masukan syntax **node index**:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
D:\25-26 Genap\Pem Web\wokspaces\MERN-STACK\backend>node index  
Server running on port 3000
```

Jika menampilkan informasi 'Server running on port 3000' maka backend server anda telah berhasil dibuat. Jika anda masukan localhost:3000 pada browser maka akan tampil seperti dibawah ini:



Untuk mematikan serve tekan keyboard Ctrl+C, maka server akan mati. Perlu diketahui, pada environment express setiap kali melakukan perubahan code, diharuskan untuk merestart server. Backend ini tidak dapat mengupdate perubahan secara real-time.

Jika ingin server berjalan secara real-time, dapat menginstall **library nodemon**. Namun ini hanya bekerja pada OS Windows dan tidak bisa bekerja pada ranah production. Disarankan digunakan pada saat development. Cara menjalankannya cukup masukan syntax ``nodemon index``.

Langkah-3: Integrasi Database

1. Buatlah database MySQL bernama ``pw_praktikum_db``
2. Ubahlah file index.js dengan menambahkan pengecekan koneksi ke MySQL

```

const express = require("express");
const mysql = require("mysql2");

const app = express();

app.use(express.json());

const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "pw_praktikum_db"
});

db.connect((err) => {
  if (err) {
    console.error("Koneksi ke database gagal:", err.message);
    process.exit(1);
  }
  console.log("Koneksi ke database MySQL berhasil!");
});

app.get("/", (req, res) => {
  res.send("Server berjalan");
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});

```

Penjelasan:

- Import *mysql2* untuk koneksi ke MySQL
- Membuat konfigurasi koneksi dengan *createConnection()*
- Menggunakan method *connect()* untuk mengecek koneksi
- Jika berhasil, akan muncul pesan *"Koneksi ke database MySQL berhasil!"*
- Jika gagal, akan muncul pesan error dan aplikasi akan berhenti

3. Setelah mengupdate, restart kembali servernya lalu buatlah beberapa kondisi seperti, koneksi ke MySQL nyala, matikan atau nama database tidak ditemukan.

```

D:\25-26 Genap\Pem Web\workspaces\MERN-STACK\backend>node index
Server running on port 3000
Koneksi ke database gagal:

```

Gambar Koneksi server gagal ke database

```

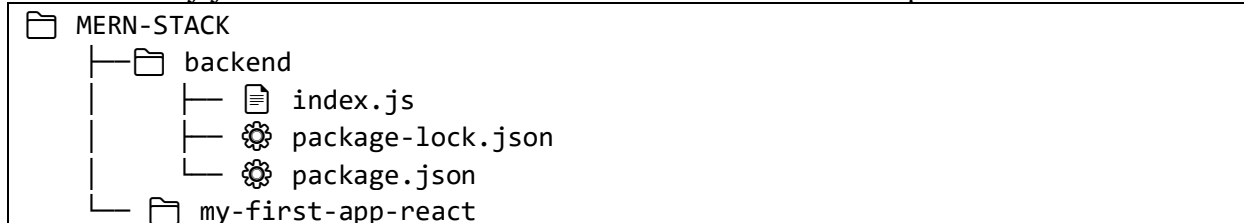
D:\25-26 Genap\Pem Web\workspaces\MERN-STACK\backend>node index
Server running on port 3000
Koneksi ke database MySQL berhasil!

```

Gambar Koneksi server berhasil ke database

Langkah-4: Membangun Frontend dengan React

Selanjutnya membuat project web client menggunakan framework React, dimana project yang akan dibuat sejajar atau masih berada dalam folder MERN-STACK seperti ilustrasi dibawah ini.



1. Buka terminal dan arahkan ke direktori path MERN-STACK lalu masukan syntax untuk membuat project react :

```
npx create-next-app@latest
```

Pada terminal anda akan ditanya beberapa instruksi seperti membuat nama project hingga template project. Isilah pertanyaan tersebut sesuai dengan instruksi dibawah ini:

```
D:\25-26 Genap\Pem Web\wokspaces\MERN-STACK>npx create-next-app@latest
Need to install the following packages:
create-next-app@16.1.6
Ok to proceed? (y) y
✓ What is your project named? .. my-first-app-react
✓ Would you like to use the recommended Next.js default? ... Yes
✓ Would you like to use TypeScript? ... No / Yes
✓ Which linter would you like to use? » ESLint
✓ Would you like to use React Compiler? ... No / Yes
✓ Would you like to use Tailwind CSS? ... No / Yes
✓ Would you like your code inside a `src/` directory? ... Yes
✓ Would you like to use App Router? (recommended) ... Yes
✓ Would you like to customize the import alias (`@/*` by default)? ... No / Yes
✓ What import alias would you like configured? ... @/*
Creating a new Next.js app in D:\25-26 Genap\Pem Web\wokspaces\MERN-STACK\my-first-app-react.
```

Masukan nama project:
my-first-app-react

Pilih No

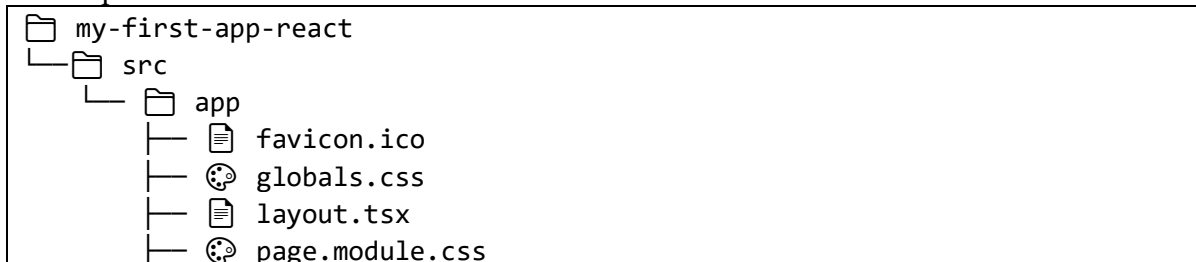
Jika sudah mengisi selanjutnya akan menginstalasi kebutuhan dari project tersebut. Setelah selesai akan menampilkan informasi **“Success! Created”** pada terminal anda, menandakan project react telah berhasil dibuat.

```
Generating route types...
✓ Types generated successfully

Initialized a git repository.

Success! Created my-first-app-react at D:\25-26 Genap\Pem Web\wokspaces\MERN-STACK\my-first-app-react
```

2. Bukalah project dengan IDE dan masuk ke dalam file page.tsx yang berada pada src/app/ lalu modifikasi file tersebut dengan menggantinya menjadi kode sederhana React untuk menampilkan “Hello World”.



File *page.tsx*:

```
import type { Metadata } from "next";

export const metadata: Metadata = {
  title: "My First Project"
};

export default function Home() {
  return (
    <div>
      <h1>Hello World!!</h1>
    </div>
  );
}
```

3. Selanjutnya menjalankan project react tersebut dengan memasukan syntax `npm run dev` pada terminal project `'my-first-app-react'`.

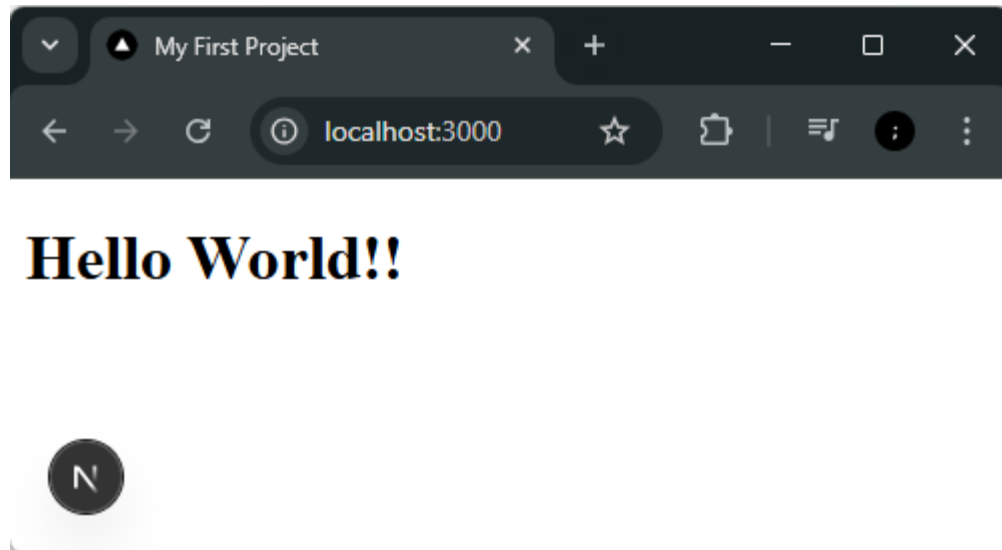
D:\25-26 Genap\Pem Web\wokspaces\MERN-STACK\my-first-app-react>npm run dev

```
> my-first-app-react@0.1.0 dev
> next dev
```

```
▲ Next.js 16.1.6 (Turbopack)
- Local:      http://localhost:3000
- Network:    http://192.168.1.11:3000
```

```
✓ Starting...
✓ Ready in 1789ms
GET / 200 in 1572ms (compile: 839ms, render: 733ms)
```

Buka browser dan masukan local network sesuai yang disampaikan oleh terminal, yaitu <http://localhost:3000> atau menggunakan ip public <http://192.168.1.11:3000>



Gambar Halaman Utama Project Frontend menggunakan React Framework

Untuk penjelasan lebih lanjut mengenai ruanglingkup kebutuhan Framework React akan dijelaskan pada bab selanjutnya.

Langkah-5: Integrasi Frontend dan Backend

Pada Langkah 3, telah berhasil membangun backend sederhana menggunakan Express.js yang berjalan di atas lingkungan runtime Node.js. Backend tersebut berfungsi sebagai penyedia layanan (service) atau REST API yang menangani request dan response, serta mengelola komunikasi dengan database.

Selanjutnya, pada Langkah 4, telah membangun frontend menggunakan React sebagai antarmuka pengguna (user interface). Frontend ini bertugas menampilkan data kepada pengguna serta mengirimkan request ke backend melalui HTTP.

Sebelum melakukan proses integrasi antara frontend dan backend, terdapat hal penting yang perlu dipahami, yaitu terkait penggunaan port pada local environment.

Dalam arsitektur *client-server*, setiap aplikasi yang berjalan pada komputer lokal (*localhost*) menggunakan nomor port tertentu sebagai jalur komunikasi. Port berfungsi sebagai “alamat pintu” yang membedakan satu layanan dengan layanan lainnya dalam satu sistem yang sama.

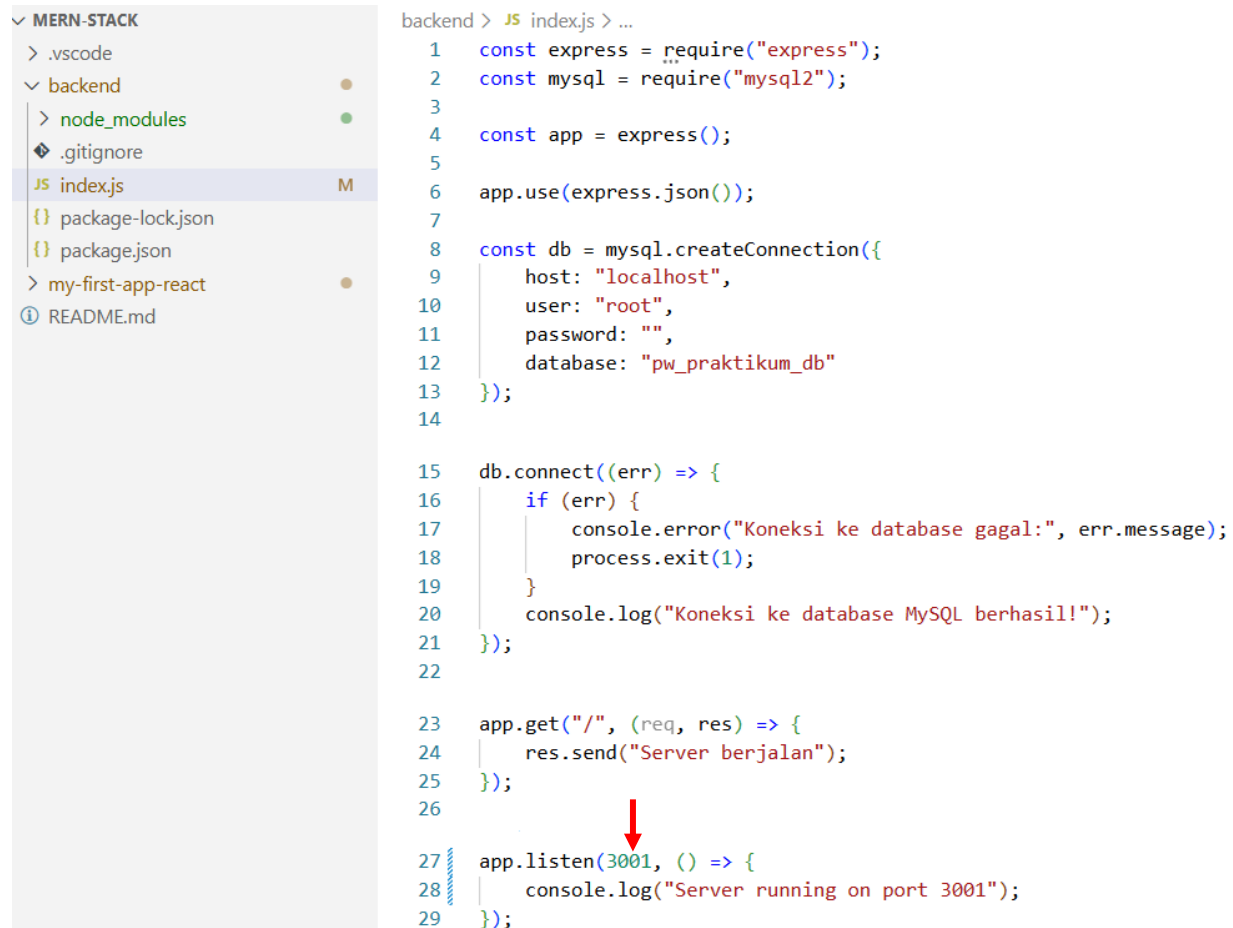
Karena backend dan frontend dijalankan secara bersamaan pada satu mesin (*local development*), maka keduanya **tidak boleh** menggunakan nomor port yang sama. Jika dua aplikasi mencoba berjalan pada port yang identik (misalnya sama-sama menggunakan port 3000), maka akan terjadi konflik dan salah satu aplikasi tidak dapat dijalankan.

Sebagai contoh:

- Backend (Express) dapat dijalankan pada `http://localhost:3001`
- Frontend (React) dapat dijalankan pada `http://localhost:3000`

Pemahaman mengenai konfigurasi port ini sangat penting sebelum proses integrasi dilakukan, karena menjadi dasar komunikasi antara frontend dan backend dalam pengembangan aplikasi *full-stack*.

1. Bukalah project MERN-STACK pada IDE seperti VS Code, lalu ubah port backend menjadi 3001



```
backend > JS index.js > ...
1  const express = require("express");
2  const mysql = require("mysql2");
3
4  const app = express();
5
6  app.use(express.json());
7
8  const db = mysql.createConnection({
9    host: "localhost",
10   user: "root",
11   password: "",
12   database: "pw_praktikum_db"
13 });
14
15 db.connect((err) => {
16   if (err) {
17     console.error("Koneksi ke database gagal:", err.message);
18     process.exit(1);
19   }
20   console.log("Koneksi ke database MySQL berhasil!");
21 });
22
23 app.get("/", (req, res) => {
24   res.send("Server berjalan");
25 });
26
27 app.listen(3001, () => {
28   console.log("Server running on port 3001");
29 });
```

Ubahlah port backend pada `app.listen` dengan parameter 3001, agar tidak sama dengan default port frontend yaitu 3000.

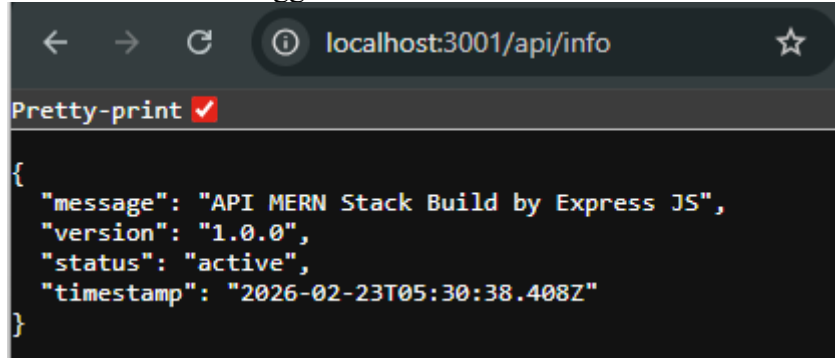
1. Membuat route sederhana untuk menampilkan Rest API

Tahap ini akan membuat sebuah url Rest API sederhana dengan address <http://localhost:3001/api/info> dimana API tersebut menampilkan informasi mengenai version dari backend.

Pada file `index.js` tambahkan code route dibawah ini:

```
app.get("/api/info", (req, res) => {
  res.json({
    message: "API MERN Stack Build by Express JS",
    version: "1.0.0",
    status: "active",
    timestamp: new Date()
  });
});
```

Selanjutnya jalankan kembali server backend expressnya untuk mengecek apakah url tersebut sudah berhasil dibuat. Contoh menggunakan browser:



Penjelasan lebih lanjut mengenai ruang lingkup kebutuhan express akan dibahas pada bab selanjutnya.

2. Memberikan akses backend agar frontend dapat berkomunikasi
Masih pada file index.js milik backend, tambahkan code CORS untuk memberikan akses local network frontend, dimana akses local network frontend adalah <http://localhost:3000>

```
const express = require("express");
const mysql = require("mysql2");
const cors = require("cors");
const app = express();

app.use(cors({
  origin: "http://localhost:3000",
  credentials: true
}));

app.use(express.json());

//rest of code
```

3. Mengintegrasikan frontend React dengan server Express
Pada **project react** yang telah dibuat tambahkan sebuah library package bernama axios dengan syntax:

```
npm install axios
```

Axios bertindak sebagai klien HTTP yang memudahkan komunikasi antara aplikasi web dan API, didukung fitur canggih seperti konversi JSON otomatis, interseptor, dan penanganan kesalahan yang lebih baik. Pengenalan lebih lanjut perihal AXIOS akan dibahas pada bab selanjutnya.

4. Memanggil Rest API untuk mengecek versioning backend
Untuk memanggil url API <http://localhost:3001/api/info> pada file *page.tsx* tambahkanlah code dibawah ini:

```

"use client";

import axios from "axios";
import { useEffect, useState } from "react";

export default function Home() {
  const [info, setInfo] = useState<any>(null);

  const GetAPIInfo = async () => {
    try {
      const response = await axios.get(
        "http://localhost:3001/api/info");
      setInfo(response.data);
    } catch (error) {
      console.error('Error fetching API data:', error);
      setInfo(error?.message);
    }
  }

  useEffect(() => {
    GetAPIInfo();
  }, []);

  return (
    <div>
      <h1>Hello World!!</h1>
      {info && (
        <pre style={{
          backgroundColor: "#f4f4f4",
          padding: "15px",
          borderRadius: "8px",
          border: "1px solid #ddd",
          overflow: "auto"
        }}>
          {JSON.stringify(info, null, 2)}
        </pre>
      )}
    </div>
  );
}

```

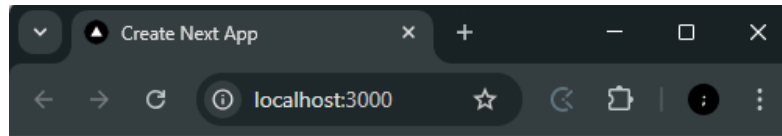
Penjelasan:

- a. **"use client"**: Menandai komponen ini sebagai *Client Component*.
- b. **axios** : Library untuk melakukan HTTP request ke backend
- c. **const [info, setInfo] = useState<any>(null)**: Membuat state info dengan nilai awal null
- d. **GetAPIInfo()** : Fungsi yang berjalan secara asynchronous
- e. **axios.get**: Melakukan HTTP GET request ke backend Express.js
- f. **useEffect Hook**: Menjalankan **GetAPIInfo()** sekali saat komponen pertama kali dimuat

5. Hasil testing Rest API pada frontend

Berikut ini adalah beberapa kondisi hasil dari integrasi Frontend dengan Backend:

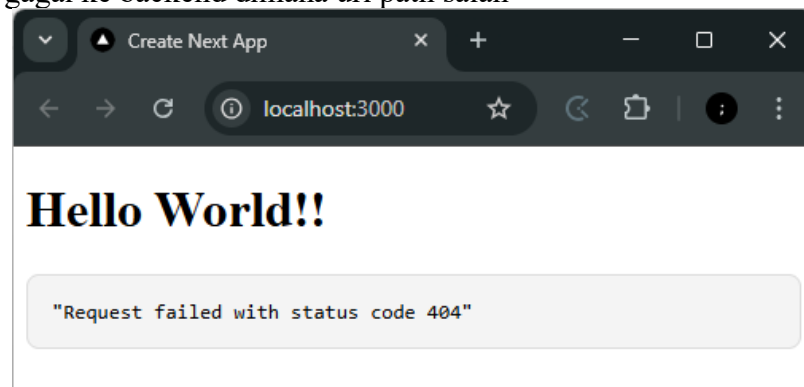
- Jika koneksi berhasil ke address <http://localhost:3001/api/info>



Hello World!!

```
{  "message": "API MERN Stack Build by Express JS",  "version": "1.0.0",  "status": "active",  "timestamp": "2026-02-23T06:16:10.955Z"}
```

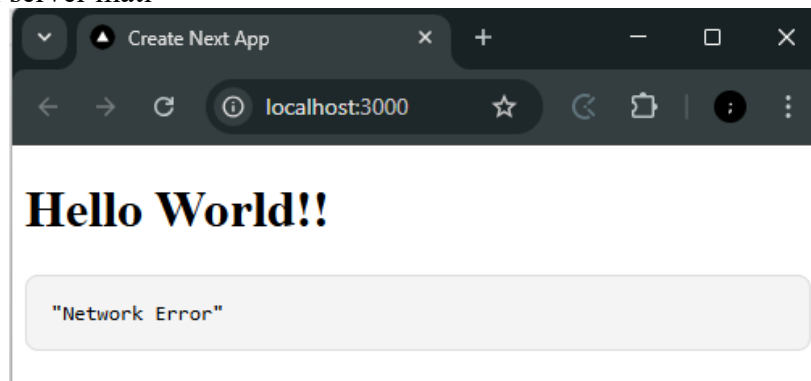
- Jika koneksi gagal ke backend dimana url path salah



Hello World!!

`"Request failed with status code 404"`

- Jika koneksi server mati



Hello World!!

`"Network Error"`

4.4 Tugas Pembelajaran

1. Buatlah dokumentasi dalam mengerjakan soal Latihan Pembelajaran, dengan memberikan Screenshot baik dari sisi aplikasi ataupun code yang sudah dimodifikasi dengan penjelasan dan alur yang baik dan benar. Dokumentasi disimpan dalam bentuk PDF dan ditaruh dalam project web yang telah anda kerjakan.
2. Upload project tersebut pada GITHUB dengan menggunakan repository bernama **"PW-MERN-STACK-NPM"** dengan nama commit **"Praktikum 4"**.

Pengumpulan Tugas Latihan Praktikum dikumpulkan selama 1 minggu kedepan sejak hari ini (sesuai dengan jadwal praktikum).